

게임 수학

강의 7: 행렬식과 역행렬

행렬식 (Determinant), 여인수 전개, 역행렬 (Inverse Matrix)

동명대학교 게임공학과
강영민

2026년 1학기

Contents

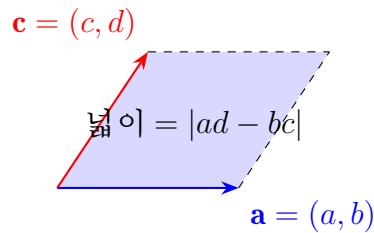
1	행렬식(Determinant)이란?	2
2	2×2 행렬식	2
3	소행렬식과 여인수	2
4	여인수 전개	3
4.1	3×3 행렬식 계산	3
4.2	사루스 공식 (Sarrus' Rule)	4
5	행렬식의 성질	4
6	역행렬 (Inverse Matrix)	5
6.1	2×2 역행렬 공식	5
6.2	여인수 행렬과 수반 행렬	5
7	가우스-조르단 소거법으로 역행렬 구하기	6
8	선형 연립방정식과 역행렬	6
9	Python 구현	7
10	연습 문제	9
11	요약	10
A	크라머 공식 (Cramer's Rule)	10

1 행렬식(Determinant)이란?

행렬식(determinant)은 정방행렬에 하나의 스칼라값을 대응시키는 함수이다. $n \times n$ 행렬 A 의 행렬식을 $\det(A)$ 또는 $|A|$ 로 표기한다.

기하적 의미. 행렬식의 절댓값은 행렬의 열벡터들이 만드는 부피(또는 넓이)와 같다:

- 2×2 : 두 열벡터가 만드는 평행사변형의 넓이
- 3×3 : 세 열벡터가 만드는 평행육면체의 부피
- 부호: 방향(오른손/왼손 좌표계)을 나타냄



2 2×2 행렬식

정의 — 2×2 행렬식

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

예제 1 — 2×2 행렬식

$\det \begin{pmatrix} 3 & 1 \\ 2 & 4 \end{pmatrix}$ 를 구하라.

$$\det = 3 \cdot 4 - 1 \cdot 2 = 12 - 2 = 10$$

두 열벡터 $(3, 2)$ 와 $(1, 4)$ 가 이루는 평행사변형의 넓이가 10이다.

3 소행렬식과 여인수

$n \times n$ 행렬의 행렬식은 더 작은 행렬의 행렬식으로 재귀적으로 계산한다.

정의 — 소행렬식 (Minor)

$n \times n$ 행렬 A 에서 i 행과 j 열을 제거하여 얻은 $(n-1) \times (n-1)$ 행렬의 행렬식을 소행렬식 M_{ij} 라 한다.

정의 — 여인수 (Cofactor)

소행렬식에 부호를 붙인 값:

$$C_{ij} = (-1)^{i+j} M_{ij}$$

부호 패턴:

$$\begin{pmatrix} + & - & + & \cdots \\ - & + & - & \cdots \\ + & - & + & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

4 여인수 전개

정리 — 여인수 전개 (Cofactor Expansion)

$n \times n$ 행렬 A 의 행렬식은 임의의 행 i 또는 열 j 를 따라 전개할 수 있다:
 i 번째 행에 대한 전개:

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij} = \sum_{j=1}^n a_{ij} (-1)^{i+j} M_{ij}$$

j 번째 열에 대한 전개:

$$\det(A) = \sum_{i=1}^n a_{ij} C_{ij}$$

4.1 3×3 행렬식 계산

예제 2 — 3×3 행렬식

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

$\det(A)$ 를 첫 번째 행으로 여인수 전개하여 계산하라.

$$\begin{aligned} \det(A) &= 1 \cdot (+1) \det \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix} + 2 \cdot (-1) \det \begin{pmatrix} 4 & 6 \\ 7 & 9 \end{pmatrix} + 3 \cdot (+1) \det \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} \\ &= 1(45 - 48) - 2(36 - 42) + 3(32 - 35) \\ &= 1(-3) - 2(-6) + 3(-3) \\ &= -3 + 12 - 9 = 0 \end{aligned}$$

$\det(A) = 0$ 이므로 이 행렬은 특이 행렬(singular)이다 (역행렬이 없다). (실제로 3행 = 2×2행 - 1행이므로 선형 종속)

4.2 사루스 공식 (Sarrus' Rule)

3×3 행렬식을 빠르게 계산하는 패턴:

$$\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bfg + cdh - ceg - afh - bdi$$

사루스 공식 기억법

대각선 방향으로 내려가며 곱한 것을 더하고(+), 반대 대각선 방향으로 내려가며 곱한 것을 뺀다(-). 단, 4×4 이상에서는 사루스 공식이 통하지 않는다.

5 행렬식의 성질

정리 — 행렬식의 주요 성질

$A, B: n \times n$ 정방행렬, $k \in \mathbb{R}$:

1. $\det(I) = 1$
2. $\det(A^T) = \det(A)$
3. $\det(AB) = \det(A) \det(B)$
4. $\det(kA) = k^n \det(A)$
5. $\det(A^{-1}) = 1/\det(A)$ (역행렬이 존재할 때)
6. 두 행을 바꾸면 부호가 바뀐다.
7. 한 행이 다른 행의 스칼라 배이면 $\det = 0$.
8. 삼각 행렬의 행렬식 = 대각 원소의 곱.

성질 8 — 삼각 행렬.

$$\det \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{pmatrix} = a_{11} \cdot a_{22} \cdot a_{33}$$

이는 여인수 전개를 반복할 필요 없이 $O(n)$ 에 계산할 수 있음을 뜻한다. LU 분해가 이 성질을 활용한다.

예제 3 — 행렬식 성질 활용

$\det(A) = 3, \det(B) = -2$ 일 때 다음을 구하라: (a) $\det(AB)$, (b) $\det(2A)$ (3×3), (c) $\det(A^T)$

- (a) $\det(AB) = \det(A) \det(B) = 3 \cdot (-2) = -6$
 (b) $\det(2A) = 2^3 \det(A) = 8 \cdot 3 = 24$
 (c) $\det(A^T) = \det(A) = 3$

6 역행렬 (Inverse Matrix)

정의 — 역행렬

$n \times n$ 정방행렬 A 의 역행렬 A^{-1} 은 다음을 만족한다:

$$AA^{-1} = A^{-1}A = I$$

역행렬이 존재할 조건: $\det(A) \neq 0$ (비특이 행렬, non-singular matrix).

6.1 2×2 역행렬 공식

정리 — 2×2 역행렬

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \det(A) = ad - bc \neq 0$$

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

기억법: 대각을 바꾸고($a \leftrightarrow d$), 나머지에 부호를 바꾼 후($b \rightarrow -b, c \rightarrow -c$), 행렬식으로 나눈다.

예제 4 — 2×2 역행렬

$A = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$ 의 역행렬을 구하라.

$$\det(A) = 2 \cdot 3 - 1 \cdot 5 = 6 - 5 = 1$$

$$A^{-1} = \frac{1}{1} \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix} = \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix}$$

$$\text{검증: } AA^{-1} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix} \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix} = \begin{pmatrix} 6-5 & -2+2 \\ 15-15 & -5+6 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \checkmark$$

6.2 여인수 행렬과 수반 행렬

정의 — 수반 행렬 (Adjugate Matrix)

행렬 A 의 여인수 행렬 C 는 (i, j) 원소가 여인수 C_{ij} 인 행렬이다. 수반 행렬(adjugate) $\text{adj}(A) = C^T$ (여인수 행렬의 전치).

정리 — 수반 행렬을 이용한 역행렬

$$A^{-1} = \frac{1}{\det(A)} \cdot \text{adj}(A) = \frac{1}{\det(A)} C^T$$

예제 5 — 3×3 역행렬

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

의 역행렬을 구하라.

Step 1: $\det(A)$ 계산 (첫 번째 행 전개):

$$\begin{aligned} \det(A) &= 1 \cdot \det \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix} - 0 + 1 \cdot \det \begin{pmatrix} 0 & 2 \\ 1 & 1 \end{pmatrix} \\ &= 1(0 - 1) + 1(0 - 2) = -1 - 2 = -3 \end{aligned}$$

Step 2: 여인수 계산: $C_{11} = (2 \cdot 0 - 1 \cdot 1) = -1$, $C_{12} = -(0 \cdot 0 - 1 \cdot 1) = 1$,
 $C_{13} = (0 \cdot 1 - 2 \cdot 1) = -2$
 $C_{21} = -(0 \cdot 0 - 1 \cdot 1) = 1$, $C_{22} = (1 \cdot 0 - 1 \cdot 1) = -1$, $C_{23} = -(1 \cdot 1 - 0 \cdot 1) = -1$
 $C_{31} = (0 \cdot 1 - 1 \cdot 2) = -2$, $C_{32} = -(1 \cdot 1 - 0 \cdot 1) = -1$, $C_{33} = (1 \cdot 2 - 0 \cdot 0) = 2$

Step 3:

$$A^{-1} = \frac{1}{-3} \begin{pmatrix} -1 & 1 & -2 \\ 1 & -1 & -1 \\ -2 & -1 & 2 \end{pmatrix}^T = \frac{1}{-3} \begin{pmatrix} -1 & 1 & -2 \\ 1 & -1 & -1 \\ -2 & -1 & 2 \end{pmatrix}$$

(이 경우 여인수 행렬이 대칭이므로 전치 불필요)

7 가우스-조르단 소거법으로 역행렬 구하기

수반 행렬 방법은 이론적이지만 n 이 클수록 비효율적이다. 실용적으로는 가우스-조르단 소거법을 사용한다.

가우스-조르단 역행렬 알고리즘

$(A|I)$ 를 확장 행렬로 만든 후, 행 연산(row operation)으로 좌측을 I 로 변환:

$$(A|I) \xrightarrow{\text{행 연산}} (I|A^{-1})$$

허용되는 행 연산:

1. 두 행 교환: $R_i \leftrightarrow R_j$
2. 행에 스칼라 배: $R_i \leftarrow kR_i$ ($k \neq 0$)
3. 다른 행의 스칼라 배를 더함: $R_i \leftarrow R_i + kR_j$

8 선형 연립방정식과 역행렬

역행렬의 가장 중요한 활용은 선형 연립방정식 풀기이다.

정리 — 연립방정식의 행렬 표현

연립방정식 $A\mathbf{x} = \mathbf{b}$ 에서 $\det(A) \neq 0$ 이면:

$$\mathbf{x} = A^{-1}\mathbf{b}$$

예제 6 — 연립방정식 풀기

$2x + y = 5, 5x + 3y = 13$ 을 행렬로 풀어라.

$$A = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} 5 \\ 13 \end{pmatrix}$$

예제 4에서 $A^{-1} = \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix}$

$$\mathbf{x} = A^{-1}\mathbf{b} = \begin{pmatrix} 3 & -1 \\ -5 & 2 \end{pmatrix} \begin{pmatrix} 5 \\ 13 \end{pmatrix} = \begin{pmatrix} 15 - 13 \\ -25 + 26 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

$$x = 2, y = 1.$$

9 Python 구현

실습

```
1 import numpy as np
2
3 # =====
4 # 1. 행렬식계산
5 # =====
6 A2 = np.array([[3, 1], [2, 4]], dtype=float)
7 print("det(A2) =", np.linalg.det(A2))    # 10.0
8
9 A3 = np.array([[1, 2, 3],
10                [4, 5, 6],
11                [7, 8, 9]], dtype=float)
12 print("det(A3) =", np.linalg.det(A3))    # ~0.0 특이행렬()
13
14 # 2x2 수동구현
15 def det2(A):
16     return A[0,0]*A[1,1] - A[0,1]*A[1,0]
17
18 # 재귀적여인수전개구현
19 def minor(A, i, j):
20     rows = [r for r in range(len(A)) if r != i]
21     cols = [c for c in range(len(A[0])) if c != j]
22     return [[A[r][c] for c in cols] for r in rows]
23
24 def determinant(A):
25     n = len(A)
26     if n == 1:
27         return A[0][0]
```

```

28     if n == 2:
29         return A[0][0]*A[1][1] - A[0][1]*A[1][0]
30     result = 0
31     for j in range(n):
32         sign = (-1) ** j
33         result += sign * A[0][j] * determinant(minor(A, 0, j))
34     return result
35
36 A_list = [[1,2,3],[0,4,5],[1,0,6]]
37 print("재귀 det =", determinant(A_list))
38 print("numpy det =", np.linalg.det(np.array(A_list, dtype=float)
39 ))
40 # =====
41 # 2. 역행렬계산
42 # =====
43 A = np.array([[2, 1], [5, 3]], dtype=float)
44 A_inv = np.linalg.inv(A)
45 print("\nA(-1) =\n", A_inv)
46
47 # 검증
48 print("A @ A(-1) =\n", np.round(A @ A_inv, 10)) # I
49
50 # =====
51 # 3. 연립방정식풀기
52 # =====
53 A = np.array([[2, 1], [5, 3]], dtype=float)
54 b = np.array([5, 13], dtype=float)
55
56 # 방법 1: 역행렬
57 x1 = np.linalg.inv(A) @ b
58 print(f"\역행렬n 방법: x={x1}") # [2. 1.]
59
60 # 방법 2: numpy.linalg.solve 더 ( 수치적으로안정 )
61 x2 = np.linalg.solve(A, b)
62 print(f"solve 방법: x={x2}") # [2. 1.]
63
64 # =====
65 # 4. 행렬식성질검증
66 # =====
67 A = np.array([[1,2],[3,4]], dtype=float)
68 B = np.array([[5,6],[7,8]], dtype=float)
69
70 detA = np.linalg.det(A)
71 detB = np.linalg.det(B)
72 detAB = np.linalg.det(A @ B)
73
74 print(f"\ndet(A)={detA:.4f}, det(B)={detB:.4f}")
75 print(f"det(A)*det(B)={detA*detB:.4f}")
76 print(f"det(AB)={detAB:.4f}")

```



```

77 print(f"성질 검증: {np.isclose(detA*detB, detAB)}") # True
78
79 # 삼각행렬의행렬식 = 대각원소의곱
80 T = np.array([[2,3,4],[0,5,6],[0,0,7]], dtype=float)
81 print(f"\삼각행렬n det = {np.linalg.det(T):.4f}") # 70.0
82 print(f"대각 원소의곱 = {2*5*7}") # 70
83
84 # =====
85 # 5. 특이행렬감지조건수 ( 활용 )
86 # =====
87 def is_invertible(A, tol=1e-10):
88     return abs(np.linalg.det(A)) > tol
89
90 A_singular = np.array([[1,2],[2,4]], dtype=float)
91 A_regular = np.array([[1,2],[3,4]], dtype=float)
92 print(f"\특이n 행렬역행렬가
    능 ? {is_invertible(A_singular)}") # False
93 print(f"일반 행렬역행렬가
    능 ? {is_invertible(A_regular)}") # True

```

10 연습 문제

연습 문제

1. 다음 행렬식을 계산하라:

$$\det \begin{pmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \\ -1 & 0 & 4 \end{pmatrix}$$

2. $A = \begin{pmatrix} 1 & 2 \\ 3 & 7 \end{pmatrix}$ 의 역행렬을 공식으로 구하고, $AA^{-1} = I$ 임을 검증하라.

3. 다음 연립방정식을 행렬 방법으로 풀어라:

$$\begin{aligned} x + 2y + z &= 8 \\ -x + y + 2z &= 2 \\ 2x - y - z &= -1 \end{aligned}$$

4. $\det(A) = 5$ 이고 A 가 4×4 행렬일 때 $\det(3A)$ 를 구하라.
5. 직교 행렬(회전 행렬) R 에 대해 $\det(R) = \pm 1$ 임을 보여라. (힌트: $\det(R^T R) = \det(I) = 1$ 과 $\det(R^T) = \det(R)$ 이용)
6. Python으로 가우스-조르단 소거법을 구현하여 예제 5의 행렬의 역행렬을 구하라.

11 요약

핵심 정리

1. 행렬식: $\det(A)$ 는 열벡터들이 만드는 부피(넓이); $\det(A) = 0$ 이면 역행렬 없음.
2. 2×2 공식: $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$.
3. 여인수 전개: $\det(A) = \sum_j a_{ij}(-1)^{i+j} M_{ij}$ (재귀적).
4. 삼각 행렬: $\det =$ 대각 원소의 곱.
5. 역행렬: $A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$, 또는 가우스-조르단.
6. 연립방정식: $A\mathbf{x} = \mathbf{b}$ 의 해 $\mathbf{x} = A^{-1}\mathbf{b}$ (실용적으로는 `np.linalg.solve`).
7. Python: `np.linalg.det(A)`, `np.linalg.inv(A)`, `np.linalg.solve(A,b)`.

A 크라머 공식 (Cramer's Rule)

$A\mathbf{x} = \mathbf{b}$ 의 해를 행렬식만으로 표현하는 공식:

$$x_j = \frac{\det(A_j)}{\det(A)}$$

여기서 A_j 는 A 의 j 번째 열을 $\mathbf{b}$ 로 교체한 행렬이다.

크라머 공식은 이론적으로 중요하지만, 계산 복잡도가 $O(n! \cdot n)$ 이므로 대규모 시스템에서는 사용하지 않는다. 실용적으로는 LU 분해($O(n^3)$)를 사용한다.